Introduction to DIABLO

Data without Borders - UVRI

Mikhael Dito Manurung, MD/PhD

Leiden University Center for Infectious Diseases (LU-CID)

Leiden University Medical Center

Roadmap

- What is DIABLO
- Model tuning and evaluation
- Visualization and interpretation
- Pitfalls and practical tips

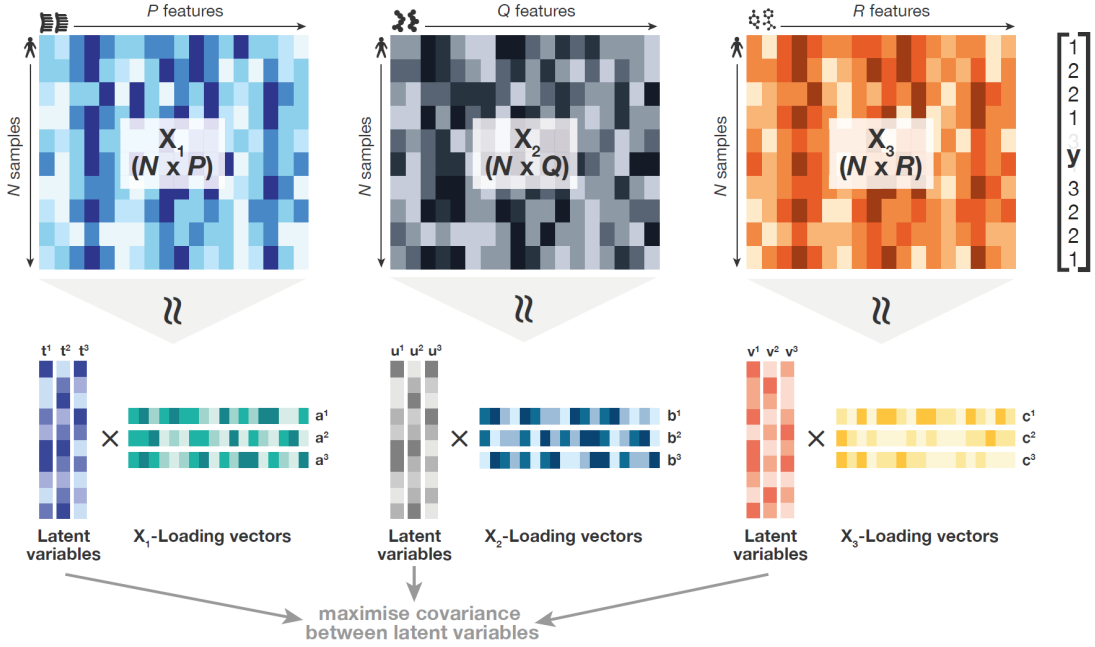
The multi-omics challenge

← All measured on the same 150 patients →

- Analyzing each block separately ignores cross-omics relationships
- Thousands of features per block → high-dimensional, noisy data
- **We want to find features that discriminate subtypes AND co-vary across blocks**

Data Integration Analysis for Biomarker discovery using Latent cOmponents

- Generalizes sPLS-DA to multiple measurements on the **same** samples
- Finds *latent components* that are **correlated across blocks** and **discriminant with respect to the outcome**



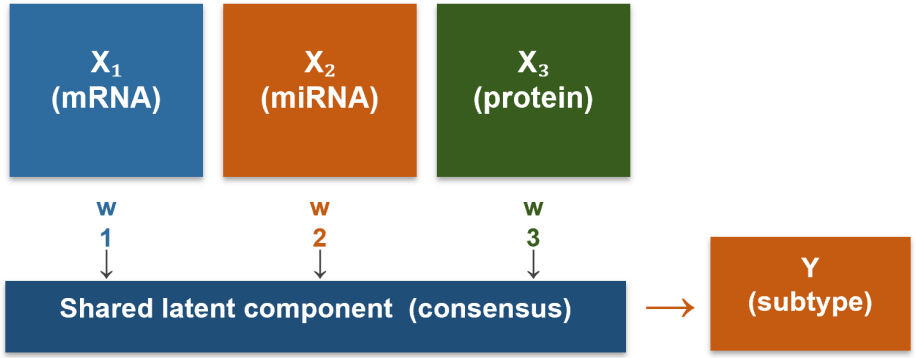
DIABLO seeks for subsets of correlated omics variables across blocks while maximizing separation between classes

Singh et al. (2019) Bioinformatics 35(17):3055–3062

Multi-block sPLS-DA

Key differences in multi-block PLS-DA:

- Each block X_k has its own weight vector w_k
- A consensus latent component is computed across all blocks
- Blocks can be weighted differently via the **design matrix**
- All blocks still predict the same outcome Y
- **DIABLO adds sparsity: only keepX features per block are retained**



Objective function of DIABLO

Maximise:

$$\Sigma\ c(j,k) \cdot \mathrm{cov}(X_j w_j, X_k w_k) + \Sigma\ \mathrm{cov}(X_k w_k, Yc)$$

Term 1: Cross-block covariance

- c(j,k) = design matrix entry for blocks j and k
- Encourages features from different blocks to co-vary
- Higher c(j,k) → stronger cross-block correlation enforced

Term 2: Outcome covariance

- Ensures each block's component discriminates Y
- This is the PLS-DA part of DIABLO
- Always present — cannot be turned off

Intuition: "The blocks talk to each other (Term 1) AND each block talks to the outcome (Term 2). The design matrix controls how loudly the blocks talk to each other."

Parameter choices

- **Design matrix (design)**
- Number of components (ncomp)
- Number of variables to select (keepX)

Design matrix: correlation between datasets

- Controls how strongly data blocks are encouraged to be correlated
- Can be based on prior knowledge or data-driven

design = 0
(Full discrimination)

0	0	0
0	0	0
0	0	0

Blocks only talk to Y.
Best classification performance.
No cross-block correlation enforced.

Recommended when prediction is the main goal

design = 0.1
(Recommended default)

0	0.1	0.1
0.1	0	0.1
0.1	0.1	0

Slight cross-block correlation.
Good balance of discrimination and integration.

Best starting point for most analyses

design = 1
(Full integration)

0	1	1
1	0	1
1	1	0

Maximum cross-block correlation.
May hurt classification.
Use only if integration is the primary goal.

Use with caution — often reduces BER

Parameter choices

Two decisions must be made before fitting the final DIABLO model:

Decision 1

ncomp

Number of latent components

- How many dimensions to extract?
- Tuned using perf() on block.plsda()
- Pick where BER stops improving

then →

Decision 2

keepX

Features to keep per block per component

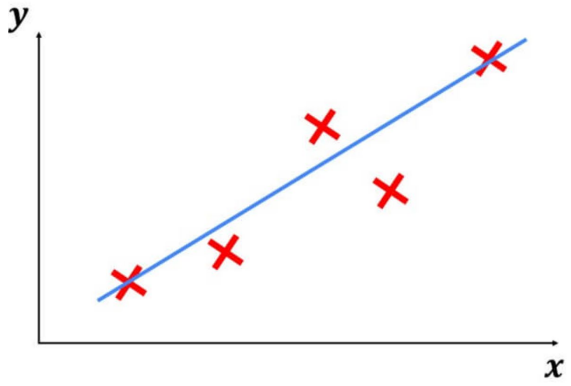
- How many features per block?
- Tuned using tune.block.splsda()
- Grid search evaluated by CV error

Two-step tuning workflow: Step 1 → fix ncomp | Step 2 → tune keepX | Step 3 → fit final model

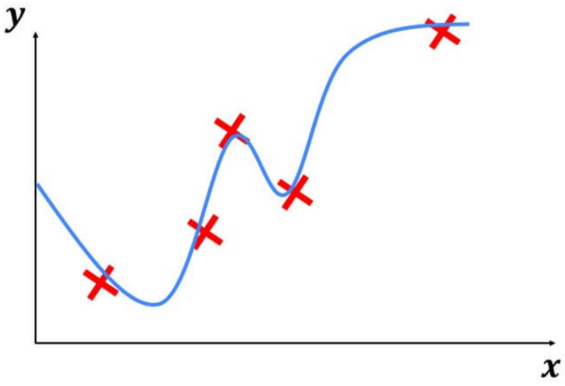
Why this order?

keepX tuning is a grid search — it fits many models. Fixing ncomp first reduces the search space dramatically. Never tune keepX and ncomp simultaneously.

Getting the right model “flexibility/fit”

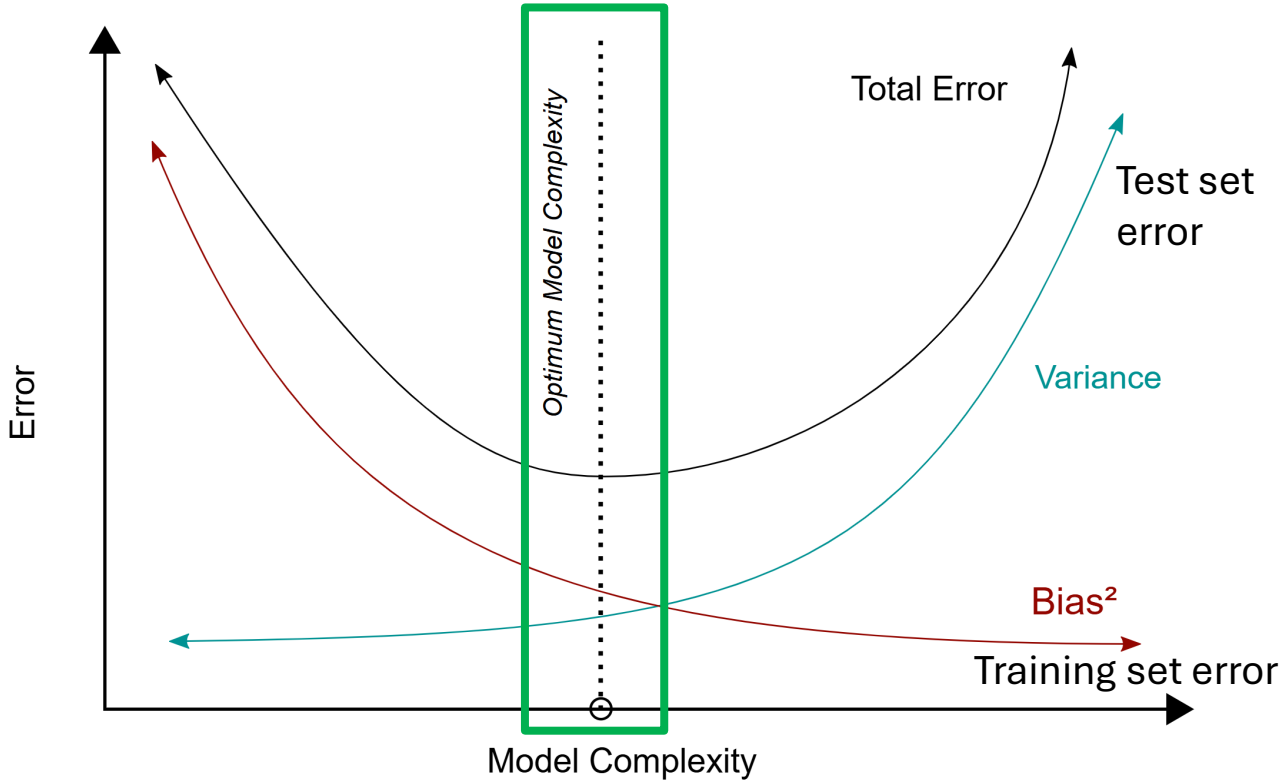


High bias underfit

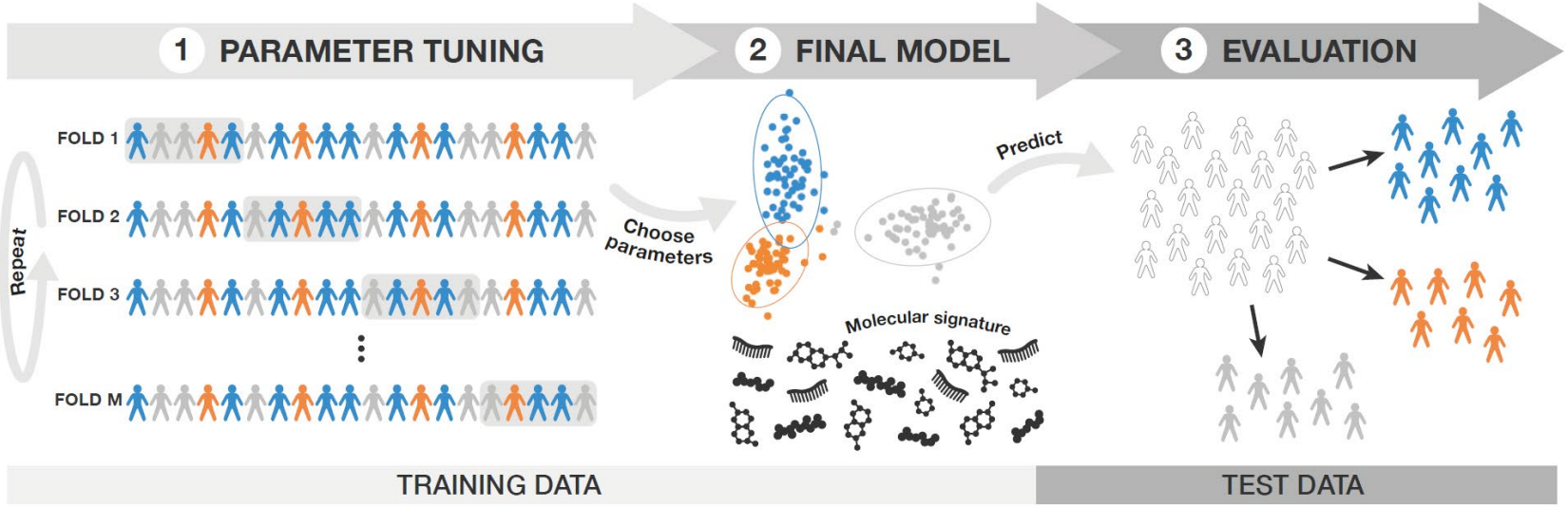


High variance overfit

Bias-variance tradeoff



Assessing model performance with cross-validation



- Ensure model does not overfit to our data and can be generalized
- Train/test split only possible in high N settings

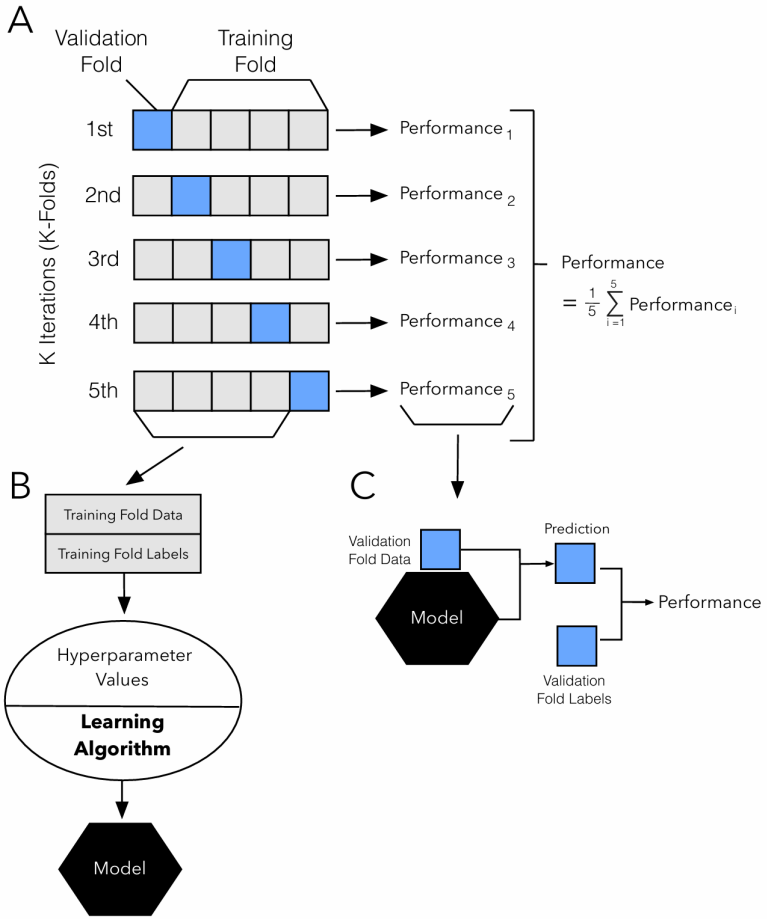
Options for cross-validation

K/M-fold cross-validation

1. **Split n samples into M equal folds (typically 10)**
2. Train DIABLO on M-1 folds, predict on the held-out fold
3. Repeat for every fold → every sample gets predicted once
4. Repeat the whole process nrepeat times with different random splits
5. Average the Balanced Error Rate (BER) across all repetitions

Balanced Error Rate (BER):

- Average error rate per class — not biased by class imbalance
- BER = 0 means perfect classification; BER = 0.5 means random guessing (2 classes)



K/M-fold cross-validation

- **M-fold cross-validation**
  - Choose M to have ~5 samples/fold
    - 10-fold for large studies, 3-5 for small
  - 50-100 repeats recommended
    - ~10 is okay for model tuning
- **Leave-one-out (LOO) CV**
  - Useful when N<10

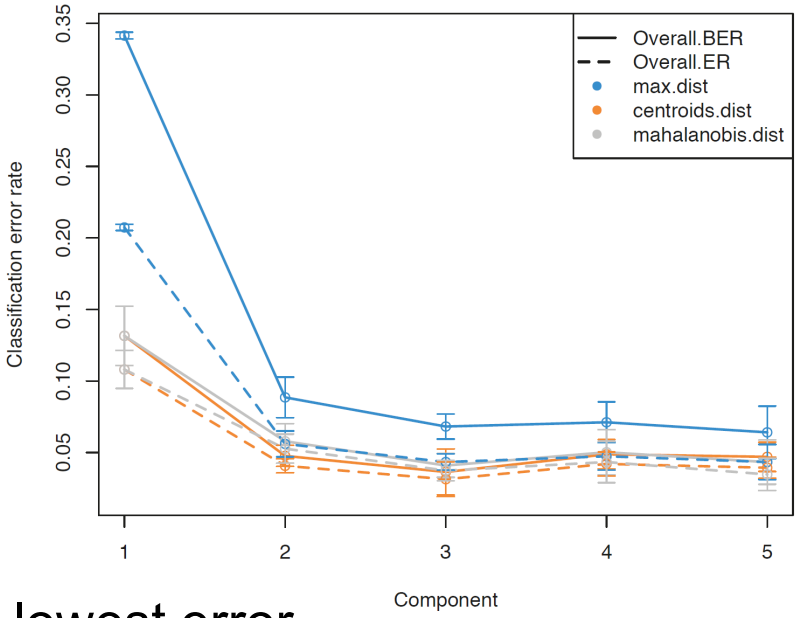
Hyperparameter tuning: ncomp and keepX

```
diablo.tcga <- block.plsda(X, Y, ncomp = 5,
design = design)

set.seed(123)
perf.diablo.tcga = perf(diablo.tcga,
validation = 'Mfold', folds = 10, nrepeat =
10)

plot(perf.diablo.tcga)
```

ncomp



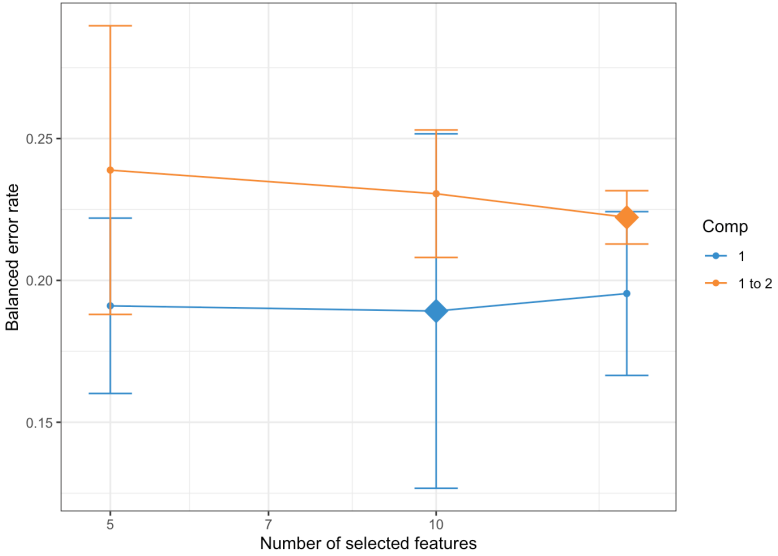
Not necessary to choose ncomp with the lowest error
(Number of class)-1 is a good start

Hyperparameter tuning: ncomp and keepX

```
test.keepX <- list(
mRNA = c(5:9, seq(10, 25, 5)),
miRNA = c(5:9, seq(10, 20, 2)),
proteomics = c(seq(5, 25, 5)))

tune.diablo.tcga <- tune.block.splsda(X, Y,
ncomp = 2, test.keepX = test.keepX, design = design,
validation = 'Mfold', folds = 10, nrepeat = 1, dist
= "centroids.dist")
```

keepX



How many features will you need for validation later?

Fitting and evaluating the final model

After tuning, fit the final model with optimal parameters:

```
# Fit final sparse multi-block PLS-DA model
final <- block.splsda(X, Y, ncomp = ncomp_opt,
                      keepX = opt_keepX,  design = design)
# opt_keepX looks like:  list(mRNA=c(15,10), miRNA=c(10,8), Protein=c(5,5))
```

Evaluate on a held-out test set:

```
predict_res <- predict(final, newdata = X_test,
                      dist = 'mahalanobis.dist')
# WeightedVote = consensus prediction across all blocks
```

⚠ **Never report training-set accuracy as model performance — always use CV error or a genuine test set**

Interpreting the output

Loading vectors (a_k)

- Weight assigned to each feature in each block
- Sparse: most are exactly 0 (not selected)
- Absolute value = importance (feature contribution)
- Sign = direction of effect relative to the component

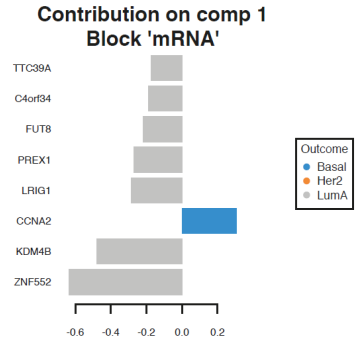
Variates / scores ($h_k = X_k a_k$)

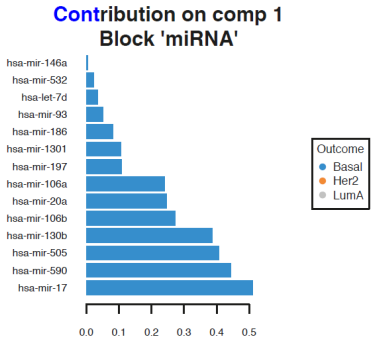
- Sample projections into the latent space (n × ncomp)
- X-axis = component 1, Y-axis = component 2
- Points close together = biologically similar samples
- Good model: classes separate clearly in this plot

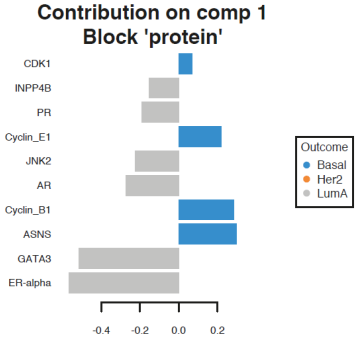
```
# Extract selected features and their loading weights
sig <- selectVar(final, block = 'mRNA', comp = 1)
sig$name    # character vector of selected gene names
sig$value   # loading magnitudes (higher = more important feature)
```

Biological interpretation: features with large absolute loadings on the same component co-vary — they define a cross-omics axis of variation linked to the outcome.

Loadings plot

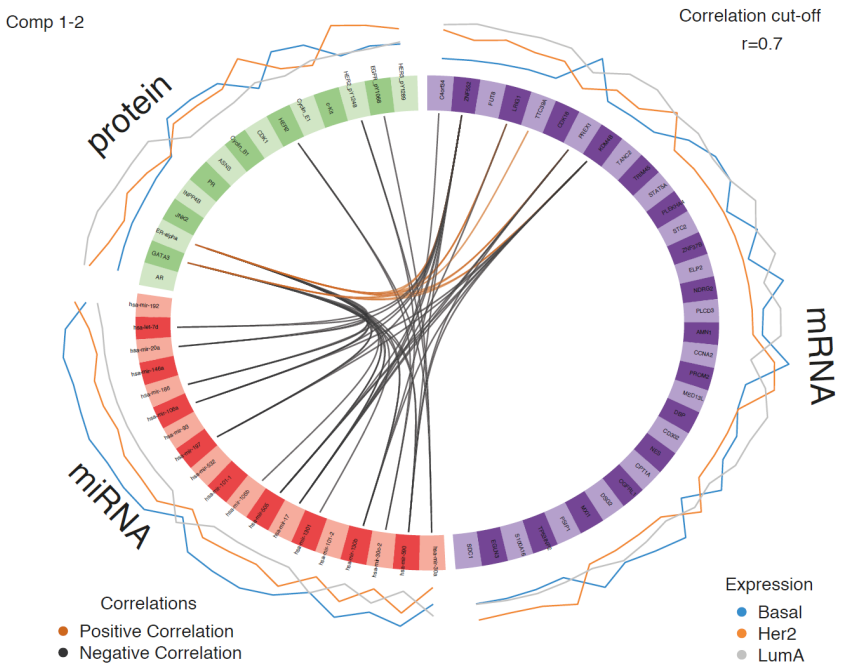




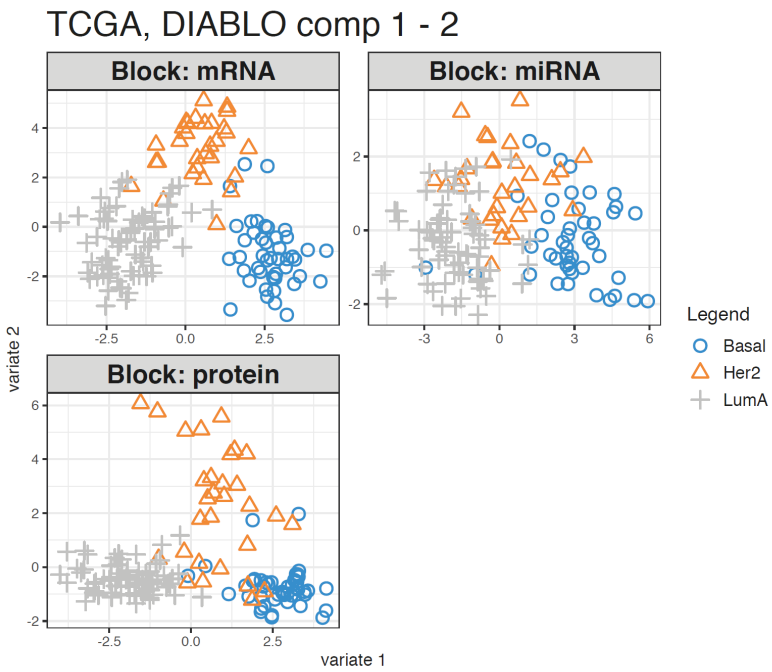


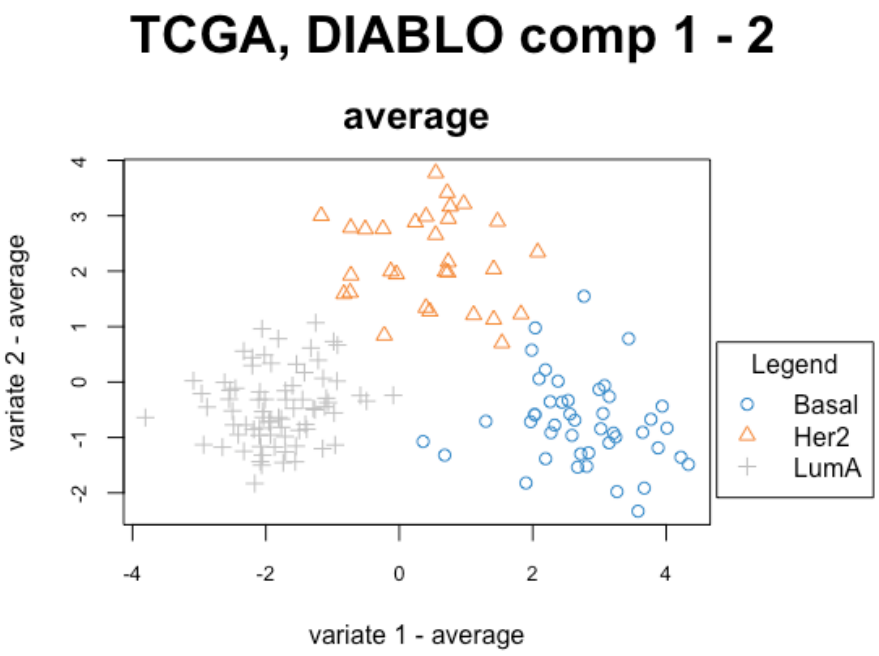
Circos plot

- Visualise cross-correlations between data types
- *Cutoff* argument can be further included to sparsify the links (preventing hairball)



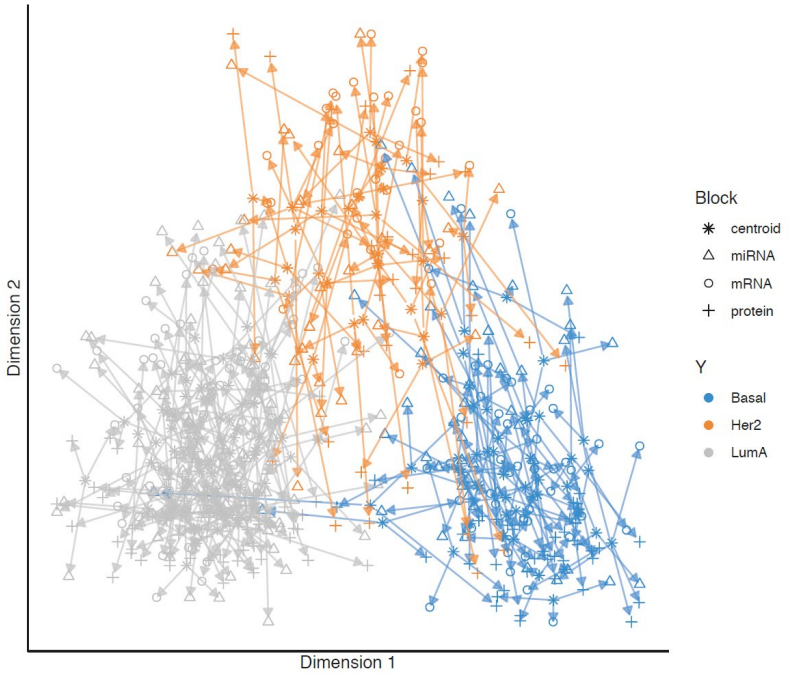
Sample plot


TCGA, DIABLO comp 1 - 2
Block: mRNA
Block: miRNA
Block: protein
variate 2
variate 1
Legend
Basal
Her2
LumA


TCGA, DIABLO comp 1 - 2
average
variate 2 - average
variate 1 - average
Legend
Basal
Her2
LumA

Arrow plot

- Highlight the agreement between all data sets
  - Arrow start: centroid
  - Arrow tip: location of the same sample but in each block



Common failure modes of DIABLO

- *N* too small
  - ~10-15 per class
- Weak correlation between datasets
  - Do not jump straight into multi-omics analysis! Do PLS check first
- High number of NA (missing) values
- **OVER-INTERPRETATION and OVER-CONFIDENCE**

Practical tips

- Always filter features before tuning
  - Variance/abundance threshold. Cut tuning time
- Use nrepeat ≥10 for stable CV estimates
- Use coarse grid first for hyperparameter tuning, then refine
- Use balanced error rate (BER) especially in class-imbalance settings
- Tune ncomp before keepX
- Keep keepX interpretable
  - 5-30 blocks/component is typical. If you want >100 features/block, why do this?
- Validate findings!
  - independent cohort/literature/wet-lab experiments

Key takeaways

- **DIABLO simultaneously discriminates groups AND integrates multiple omics blocks**
- The design matrix controls the trade-off between discrimination and integration (start with 0.1)
- Sparsity (keepX) makes results interpretable — tune it with cross-validation
- Always report CV performance (BER), not training performance

1	2	3	4	5	6
Data	**ncomp**	**keepX**	**Fit**	**Visualize**	**Interpret**
Named list X + outcome Y + design matrix	block.plsda() + perf() → pick elbow	tune.block .splsda() → grid search	block.splsda() with tuned parameters	plotIndiv plotLoadings circosPlot	Cross-block correlations & biomarkers